

KRUTRIM RAG: A Fully Offline Hybrid Retrieval-Augmented Generation Architecture for Enterprise Knowledge Retrieval and Question Answering

Kancharla Siri Lasya Reddy¹, Aesha Patel², Meetkumar Patel³

¹Department of Artificial Intelligence and Machine Learning, Chaitanya Deemed to be University, India

²Department of Information Technology, Sardar Patel College of Engineering, Gujarat, India

³Department of Information Technology, G H Patel College of Engineering and Technology, Gujarat, India

Internship Research Conducted at: Space Applications Centre (SAC), Indian Space Research Organisation (ISRO), Ahmedabad, India

Research Guide: Shri Pal Karan Sanjay, Scientist, SAC-ISRO

Abstract—Retrieval-Augmented Generation (RAG) has emerged as an effective approach for enhancing Large Language Models (LLMs) through the integration of external knowledge sources during response generation. However, most existing RAG frameworks rely on cloud-based infrastructures, resulting in challenges related to data privacy, security, latency, and internet dependency. These limitations hinder their adoption in enterprise, governmental, and research environments where sensitive information requires strict access control and local processing.

This paper presents KRUTRIM RAG, a fully offline hybrid retrieval framework designed to enable secure, scalable, and context-aware knowledge retrieval without reliance on external network services. The proposed architecture combines semantic vector retrieval using Qdrant with graph-based knowledge retrieval using Neo4j, allowing the system to leverage both semantic similarity and structured relational knowledge. In addition, locally deployed embedding models and large language models ensure complete data sovereignty, while parallel load-balanced ingestion and a multi-user GPU inference load balancer optimize resource utilization and query-processing efficiency under concurrent user load.

By integrating vector search and knowledge graph reasoning within a unified offline environment, KRUTRIM RAG improves retrieval accuracy, contextual relevance, and response quality. The key contributions include a fully offline RAG architecture, synchronized dual-path ingestion with deterministic metadata alignment, automated load-balanced parallel ingestion, a multi-user inference load balancer, and validation for privacy-preserving enterprise knowledge management and question answering.

Index Terms—Retrieval-Augmented Generation (RAG), Offline RAG, Hybrid Retrieval, Large Language Models (LLMs), Semantic Search, Vector Databases, Knowledge Graphs, Qdrant, Neo4j, Enterprise Knowledge Management, Privacy-Preserving AI, Load-Balanced Ingestion, Question Answering Systems

I. INTRODUCTION

Large Language Models have significantly advanced natural language understanding and generation, yet remain limited by the knowledge available during training and often provide inaccurate responses for domain-specific or rapidly evolving information. Retrieval-Augmented Generation address this

limitation by integrating external knowledge retrieval with language generation [1], improving factual accuracy and reducing hallucinations. Most existing RAG systems rely primarily on vector-based retrieval, where documents are represented as dense embeddings and retrieved via semantic similarity search. While effective, vector retrieval fails to capture explicit relationships and structured dependencies among entities.

Enterprises and public-sector organizations accumulate large, heterogeneous knowledge corpora - technical manuals, policy documents, mission reports, and engineering specifications - that must be searched, cross-referenced, and synthesized accurately. Effective retrieval in such settings demands more than lexical or surface-semantic matching: it requires capturing procedural structure, numerical specifications, cross-document relationships, and the provenance of every answer. Conventional dense retrieval represents documents as fixed-length embeddings and returns semantically similar passages, but treats the corpus as a bag of independent vectors, discarding the explicit relationships among entities, sections, and documents that domain experts rely upon when answering complex queries. As a result, single-pass vector retrieval frequently falls short on multi-hop, comparative, and table-grounded questions that require combining evidence across multiple sources.

Knowledge Graphs provide a complementary approach by representing information through interconnected entities and relationships, enabling structured and multi-hop reasoning [8], [9]; integrating vector retrieval with graph-based retrieval can therefore improve contextual understanding and retrieval effectiveness. However, most RAG frameworks depend heavily on cloud-based services for embedding generation, retrieval, and language model inference [2], introducing challenges related to data privacy, security, regulatory compliance, and operational reliability, particularly in enterprise and government environments where sensitive information must remain within secure infrastructure.

Beyond retrieval quality, the deployment context imposes

hard constraints. In defense, space, finance, and healthcare settings, sensitive data must not leave the local perimeter; internet connectivity may be unavailable or prohibited, and audit requirements demand that every retrieved fact be traceable to a concrete source. Cloud-based RAG stacks that offload embedding, search, and inference to hosted services therefore fail these requirements on the grounds of privacy, latency, availability, and regulatory compliance. A practical offline architecture must preserve state-of-the-art retrieval effectiveness while guaranteeing data sovereignty, provenance, and operational independence under resource-constrained, air-gapped hardware.

To address these challenges, this paper proposes KRUTRIM RAG, a fully offline hybrid retrieval framework that combines semantic vector retrieval using Qdrant [?] and graph-based retrieval using Neo4j [?] within a unified architecture. All embedding models and language models are deployed locally, ensuring complete data sovereignty while leveraging both semantic and relational knowledge retrieval. The major contributions of this work are:

- 1) A fully offline RAG architecture for secure enterprise environments.
- 2) A synchronized dual-path ingestion mechanism with deterministic metadata alignment between vector and graph representations.
- 3) An automated load-balanced parallel ingestion architecture for scalable knowledge base construction.
- 4) A heterogeneous multi-stream retrieval fusion strategy combining semantic search, graph traversal, hybrid BM25, and cross-encoder reranking.
- 5) Experimental validation for enterprise knowledge retrieval and question-answering applications.

The remainder of this paper is organized as follows. Section II reviews related work on RAG, semantic retrieval, and knowledge graphs. Section III presents the proposed framework and its component architecture, while Section IV details the ingestion and query-processing methodology. Section V reports the implementation details and configuration. Section VI summarizes the theoretical novelty and algorithmic contributions, and Section VII describes the experimental setup and benchmark datasets. Section VIII reports and analyzes the results, followed by an ablation study and failure analysis. Sections IX–XII present the ablation study, failure analysis, discussion, and future work, and Section XIII concludes the paper.

II. RELATED WORK

A. Retrieval-Augmented Generation and Semantic Retrieval

RAG improves factual accuracy and enables access to dynamic knowledge without retraining the underlying model [1]. Subsequent work has explored diverse retrieval strategies, including dense passage retrieval [11]; however, conventional RAG systems primarily rely on semantic similarity search, which struggles with relationship-aware reasoning and multi-hop knowledge discovery. Semantic retrieval represents documents as dense vector embeddings, and vector databases

such as Qdrant [?], FAISS [6], and Milvus have significantly improved contextual retrieval performance. The Hierarchical Navigable Small World (HNSW) algorithm [7], employed by Qdrant, enables efficient approximate nearest neighbor search in high-dimensional spaces.

Knowledge Graphs provide a complementary representation through interconnected entities and relationships. Graph databases such as Neo4j [?] support structured reasoning, entity linking, and multi-hop traversal using property graph models [8]. While graph retrieval improves explainability and relationship-aware reasoning, it is less effective when handling large volumes of unstructured textual information. Recent work has therefore explored hybrid architectures combining vector search with graph-based reasoning [9], where Reciprocal Rank Fusion (RRF) [12] merges ranked results from heterogeneous retrieval systems.

B. Offline AI and Research Gap

Advances in open-source LLMs [3] and locally deployable embedding models [14] have enabled offline AI deployments that improve privacy and operational control; however, most offline solutions focus only on local inference without integrated retrieval ecosystems combining semantic search and graph reasoning. As a result, existing approaches suffer from limitations including lack of structured reasoning, cloud dependence, limited retrieval capabilities, or insufficient privacy-sensitive support; no single framework effectively combines semantic vector retrieval, knowledge graph reasoning, local model deployment, and enterprise-grade scalability within a fully offline architecture.

C. Comparison with Related Hybrid and GraphRAG Systems

HybridRAG [18] demonstrates that joint vector and graph retrieval outperforms either strategy individually on financial documents, but relies on cloud-based OpenAI embeddings and LLMs. Practical GraphRAG [19] achieves 94% of LLM-based knowledge-graph extraction quality via dependency parsing with hybrid RRF retrieval, yet its LLM inference remains cloud-dependent. LEGO-GraphRAG [20] modularizes GraphRAG into subgraph-extraction and path-retrieval phases over Freebase without addressing offline deployment. The Enterprise Knowledge Retrieval framework [21] proposes LLM-generated metadata enrichment, but is vector-only and cloud-dependent for metadata generation. None of these systems operates entirely offline with local deployment of all databases, embedding models, NER, rerankers, and LLMs. To the best of our knowledge, KRUTRIM RAG is the first framework to integrate Qdrant-based semantic retrieval and Neo4j-based graph reasoning within a fully offline architecture for privacy-sensitive enterprise environments.

III. PROPOSED FRAMEWORK

A. Architectural Overview

KRUTRIM RAG is a fully offline hybrid retrieval framework for secure enterprise knowledge retrieval and question answering, integrating semantic vector retrieval through

Qdrant, graph-based retrieval through Neo4j, and locally deployed LLMs within a unified architecture. The system comprises two major subsystems: Knowledge Ingestion (Part A), running on port 8000, and Query Processing (Part B), running on port 9000, with an extraction server on port 8004 coordinating document processing between them and a GPU load-balancing service on port 5050 distributing generation requests across the model pool. All databases (Qdrant, Neo4j, MongoDB), embedding models, NER models, rerankers, and LLMs operate exclusively on local infrastructure with no internet dependency, ensuring complete data privacy and operational independence.

B. Component Architecture and Data Flow

The framework is decomposed into a set of cooperating services, each with a well-defined responsibility and communication boundary. Table I summarizes the primary components, their runtime ports, and responsibilities. All services communicate over internal localhost interfaces, so no external network egress is required at any point in either the ingestion or the query path.

TABLE I
COMPONENT RESPONSIBILITIES AND COMMUNICATION TOPOLOGY

Component	Port	Responsibility
Ingestion Master (Part A)	8000	Job orchestration, load-balanced worker dispatch, checkpointing
Query Server (Part B)	9000	Hybrid retrieval, fusion, reranking, SSE streaming
Extraction Server	8004	Docling/EasyOCR parsing and chunk distribution
GPU Load Balancer	5050	Model-slot allocation and multi-user generation
Qdrant	6333	HNSW vector search (proposition + section)
Neo4j	7687	Property-graph storage and multi-hop traversal
MongoDB	27017	Users, job states, chat history, provenance

The ingestion path proceeds as follows: raw documents are parsed and chunked by the extraction server, embedded into dense vectors and indexed into Qdrant, and simultaneously materialized as graph nodes and relationships in Neo4j, with a shared metadata fingerprint aligning the two representations at ingestion time. The query path then normalizes a user input, extracts entities, executes parallel vector and graph lookups, fuses and reranks the candidate pool, and streams the grounded response back to the client. Because the two stores share a deterministic metadata schema, a retrieved chunk can be traced to its exact document, page, and section, providing provenance that is preserved end-to-end. Each service is independently restartable, and local result caching combined with checkpoint files ensures that a failed worker resumes only the incomplete stage without reprocessing completed work.

C. Knowledge Ingestion and Query Processing

The ingestion subsystem transforms heterogeneous documents into semantic and relational knowledge representations. Documents are processed using Docling [15] for layout-aware text extraction, converted into dense embeddings using a locally deployed Nomic Embed Text v1.5 model [14] and stored in Qdrant, while entities and relationships extracted using GLiNER [10] are stored in Neo4j as a knowledge graph; ingestion tasks are distributed across concurrent workers by a load-balancing mechanism (Figure 1).

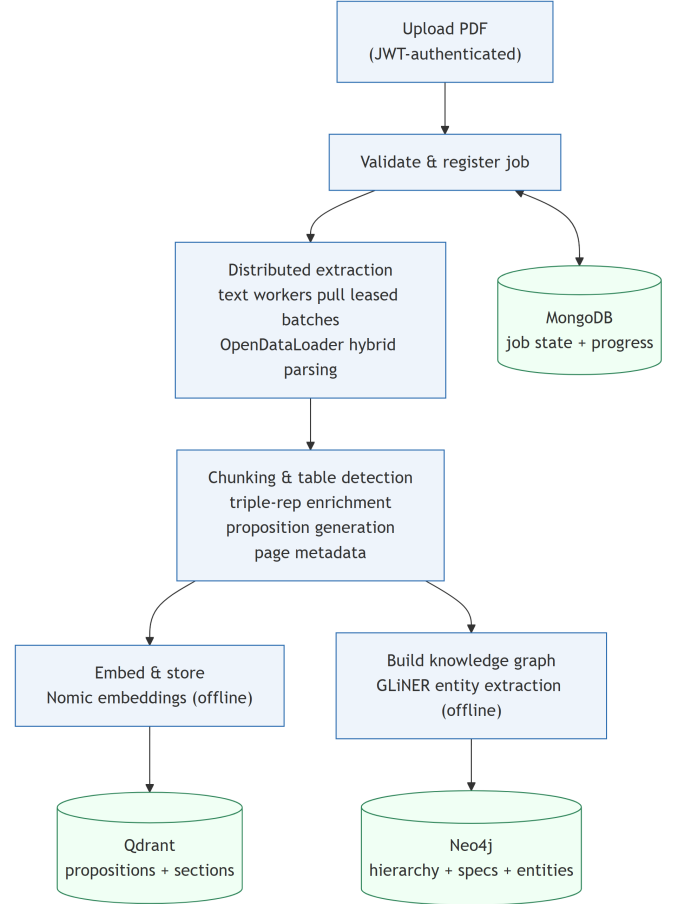


Fig. 1. Knowledge Ingestion Architecture (Part A): PDFs are extracted by distributed text workers, converted into chunks, triple representations, and atomic propositions, then ingested into Qdrant and Neo4j in parallel and fully offline.

During query processing, the query is preprocessed and analyzed using GLiNER for entity extraction; the query embedding is searched against Qdrant while extracted entities serve as anchor nodes for graph traversal in Neo4j. Retrieved candidates from both sources are combined, deduplicated, and ranked using a cross-encoder reranker (Jina Reranker v3) [13], then assembled into a unified context supplied to a locally deployed LLM (Figure 2). All retrieval and inference operations are performed locally, ensuring complete data sovereignty

and privacy preservation while supporting relationship-aware retrieval and reduced hallucinations.

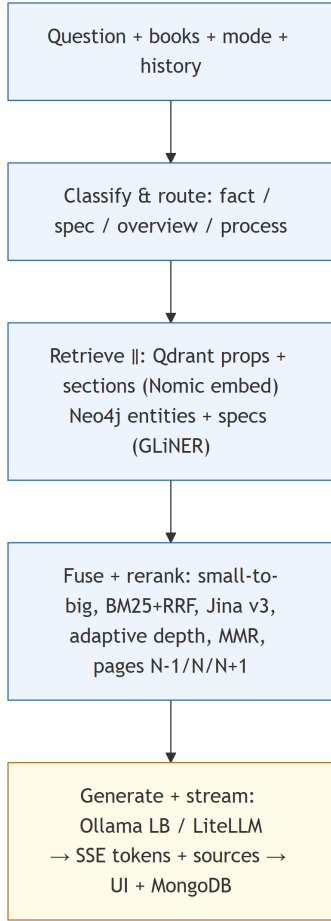


Fig. 2. Hybrid Query Processing Architecture (Part B): proposition and section searches in Qdrant run alongside GLiNER-driven entity and specification lookups in Neo4j; candidates are fused with BM25 via RRF, reranked with Jina Reranker v3 and MMR, expanded to full pages, and streamed to the chat interface through SSE.

IV. METHODOLOGY

The framework is engineered as a self-contained retrieval and synthesis system operating entirely within an air-gapped hardware infrastructure with no external network dependencies. The pipeline is divided into two sequential execution systems: the Parallelized Offline Knowledge Ingestion Pipeline (Part A) and the Hybrid Query Processing Pipeline (Part B).

A. Part A: Parallelized Offline Knowledge Ingestion

Step 1: Parallel Document Ingestion. Raw heterogeneous documentation (PDF books, technical manuals) is processed using a local, layout-aware Docling [15] extraction layer. The Master Server instantiates an automated load balancer that chunks incoming document queues and distributes them across an isolated pool of concurrent worker processes ($W_1, W_2, \dots, W_n$), each tracking document layouts and processing specific

page intervals concurrently. Workers poll the extraction server (port 8004) for jobs; three worker types form the distributed fleet: text extraction, Qdrant ingestion, and Neo4j ingestion workers. Local result caching writes completed results to disk before submission and replays them on restart, ensuring zero data loss on worker crashes or network partitions, while scanned documents are handled via a per-job OCR pipeline (EasyOCR). The parallel streams are merged into a single consolidated Markdown format preserving native headings, tables, and hierarchical boundaries.

Step 2: Micro-Chunking and Metadata Synthesis. The consolidated Markdown repository is segmented into text passages using a semantic boundary strategy driven by document heading hierarchy rather than character divisions. Each chunk captures a coherent section along with its hierarchical section path (e.g., ["3. Propulsion", "3.2 Vikas Engine"]), page range, chunk type (text or table), and a deterministic identifier derived via UUID5 from the book ID and section path. Simultaneously, a structured metadata payload ensures strict provenance tracking:

$$M(d_i) = \{\text{DocID} : \alpha, \text{Source} : \beta, \text{Page} : \gamma, \text{Section} : \delta\}$$

guaranteeing that any retrieved chunk can be verified to its exact point of origin during context assembly. Table chunks are decomposed into three representations: `structured_json` (headers and row arrays), `linearized_text` (rows as natural language), and `original_markdown`, making tables accessible to both embedding-based and structured retrieval. Atomic propositions are generated via NLTK sentence tokenization (minimum 8 words, 30 characters), each retaining a reference to its parent section for small-to-big retrieval.

Step 3: Vector Embedding and Storage. Each chunk is encoded into a 768-dimensional vector using `nomic-embed-text-v1.5` [14], selected for its 8192-token context window, low memory footprint (0.67 GB), and offline suitability (Table II); GPU acceleration (Nvidia RTX 5060) reduced embedding time by approximately 62% versus CPU:

$$\vec{d}_i = F_{\text{Nomic}}(d_i) \in \mathbb{R}^{768}$$

TABLE II
TECHNICAL PROFILE OF OFFLINE EMBEDDING MODELS

Model	Dims	Context	Memory (GB)
nomic-embed-text-v1.5	768	8192	0.67
all-MiniLM-L6-v2	384	256	0.22
BAAI/bge-base-en-v1.5	768	512	1.33
intfloat/e5-mistral-7b-instruct	4096	32768	14.0

Vectors are upserted in parallel batches into Qdrant, indexed using HNSW with cosine similarity:

$$\text{Sim}(q, d_i) = \frac{q \cdot d_i}{\|q\|_2 \|d_i\|_2}$$

The pipeline maintains two collections: a *Proposition Collection* of atomic facts for precise retrieval and a *Section*

Collection of full sections for rich context. For deployment planning, storage is estimated at approximately 3,600 bytes per vector (768 dimensions $\times$ 4 bytes per float32 plus $\sim$ 528 bytes payload overhead) in Qdrant and approximately 150 bytes per node plus 50 bytes per relationship in Neo4j, with typical enterprise sets ranging from tens to hundreds of MB.

Step 4: Topological Graph Ingestion. Entity extraction uses the local GLiNER model [10] across seven default labels (equipment, metric, organization, location, identifier, concept, person), dynamically supplemented with corpus-specific labels, building a property graph $G = (V, E, W)$ populated in Neo4j through MERGE-based Cypher operations. The graph is constructed in five layers: document hierarchy (Book $\rightarrow$ Chapter $\rightarrow$ Section $\rightarrow$ Subsection), specification nodes (regex extraction of numeric-plus-unit patterns), entity-section links (GLiNER NER $\rightarrow$ MENTIONED_IN edges), sentence co-occurrence counts, and table nodes (structured JSON $\rightarrow$ TableRow entities). Table III compares offline-capable storage engines; the vector and graph engines were selected for their native on-disk persistence and HNSW/index-free adjacency support.

TABLE III
OFFLINE STORAGE ENGINE COMPARISON

Engine	Index Algorithm	Memory Risk	Offline
Qdrant	HNSW	Low (disk + memory)	Yes
FAISS	HNSW, IVF	Memory only	Library only
Milvus	HNSW, IVF, DiskANN	Distributed	Complex
Neo4j	Index-free adjacency	Low (on-disk)	Excellent
Memgraph	In-memory	High (OOM)	Moderate
FalkorDB	In-memory	High (OOM)	Moderate

Graph Engine Justification. Neo4j was selected for its Index-Free Adjacency (IFA) execution within constrained hardware boundaries: every node stores physical disk offsets to adjacent edges, bounding traversal complexity for depth k by $O(\bar{d}^k)$ where $\bar{d}$ is the average vertex out-degree, which is completely decoupled from global graph cardinality. Non-native multi-model architectures resolve edges via global secondary indexes, incurring $O(k \cdot \log |V|)$ lookup overhead with multiplicative tuple materialization, while pure in-memory engines risk fatal Out-Of-Memory failures when massive repositories are ingested. Neo4j mitigates this with an on-disk storage layout decoupled via a deterministic page cache that maps graph segments into memory dynamically.

B. Part B: Hybrid Query Processing Pipeline

I. Query Processing and Entity Extraction. A user query Q is normalized, then analyzed by the local GLiNER model to extract domain-specific entities $E_Q = \{e_1, e_2, \dots, e_n\}$ serving as anchor nodes for graph traversal, and classified by regex pattern matching into one of five types: specification lookup, process explanation, comparative analysis, overview, or general, each applying configurable retrieval overrides (boost

multiplier, page expansion range, top-N selection, context budget, cross-book search).

II. Parallel Semantic and Graph Retrieval. The query is transformed into a dense embedding and searched against Qdrant using cosine similarity, retrieving candidates from both the proposition and section collections:

$$\vec{q} = \text{embed}(Q), \quad \text{Sim}(\vec{q}, \vec{d}_i) = \frac{\vec{q} \cdot \vec{d}_i}{\|\vec{q}\| \|\vec{d}_i\|}$$

In parallel, extracted entities drive graph traversal in Neo4j with complexity $O(\bar{d}^k)$. The system additionally supports hybrid search combining vector retrieval with BM25 [17] keyword matching (index built on-demand from section payloads with hyphen-preserving tokenization) through Reciprocal Rank Fusion [12]:

$$\text{RRF}(d) = \frac{1}{k + \text{rank}_{\text{vector}}(d)} + \frac{1}{k + \text{rank}_{\text{BM25}}(d)}, \quad k = 60$$

III. Retrieval Fusion and Re-Ranking. Results from vector, graph, and hybrid retrieval are merged into a unified candidate pool, with a confidence boost applied to evidence corroborated by multiple pathways:

$$S_{\text{boost}} = S_{\text{hybrid}} + \beta \cdot I_{\text{both}}$$

where $I_{\text{both}} = 1$ when the same section is retrieved from both Qdrant and Neo4j, and 0 otherwise. A two-pass adaptive retrieval depth first retrieves at 50% of normal depth, accepting results when the top reranker score exceeds 0.15 and otherwise re-retrieving with doubled limits. Candidates are refined by Jina Reranker v3 [13], a cross-encoder providing listwise scoring with a 131K-token context, and Maximum Marginal Relevance (MMR) [16] selection ensures diversity:

$$S_{\text{final}} = \text{CrossEncoder}(q, c_i)$$

$$\text{MMR}(c_i) = 0.7 \cdot \text{score}(c_i) - 0.3 \cdot \max_{c_j \in S} \text{sim}(c_i, c_j)$$

Proposition-score fusion bridges fine-grained and section-level retrieval: each section inherits the maximum score of its child propositions, boosting sections whose atomic facts match the query:

$$S_{\text{rerank}} = S_{\text{crossencoder}} + (\text{prop_child_score} \times 0.10 \times \text{boost_mult})$$

IV. Context Assembly and Response Generation. Top-ranked candidates undergo page-level expansion using metadata JSON files, extending each candidate’s page range to neighboring pages, and are assembled into a unified context via greedy selection by value density:

$$\text{Density} = \frac{\text{rerank_score}}{\max(1, \text{len}(\text{text}))}$$

Specifications receive a fixed high density ensuring inclusion, with two-level (summary + detail) hierarchical context for overview queries and type-specific dynamic system prompts. The assembled context is supplied to a locally deployed LLM: $R = \text{LLM}(Q, C)$, applying controlled decoding parameters

TABLE IV
TECHNOLOGY STACK AND COMPONENT ROLES

Layer	Technology / Role
Service framework	FastAPI (Part A/B servers), SSE streaming for chat
Vector store	Qdrant (HNSW, cosine), prop + section collections
Graph store	Neo4j (5-layer property graph)
Central store	MongoDB (users, job states, chat history, provenance)
Parsing	Docling layout-aware extraction, EasyOCR (scanned)
NER	GLiNER multi-v2.1 with corpus-specific labels
Embedding	nomic-embed-text-v1.5 (768-d)
Reranker	Jina Reranker v3 (131K context)
LLMs	Mistral 7B (fast/balanced), Qwen 3 14B (deep), Gemma 3
Tokenization	NLTK sentence tokenization for propositions
Interface	PDF.js in-browser viewer, metadata filtering

(temperature, top-p, top-k) that reduce hallucinations and keep responses grounded in retrieved evidence.

V. Multi-User LLM Serving with GPU Load Balancing.

To serve concurrent users rather than a single request session, the framework routes generation through a dedicated GPU load-balancing service (port 5050) that pools multiple GPU servers, each hosting model slots with up to three concurrent generations per model per server. Every user query is an independent serving request: the balancer selects the server and slot with the highest available capacity for the requested inference mode (fast and balanced serving the lightweight Mistral 7B, deep serving the larger Qwen 3 14B, with Gemma 3 and other open-weight models available in the shared pool), allocates the slot, and streams generated tokens back to the client in real time. When all slots are occupied, requests enter a FIFO wait queue and are granted slots on release, up to a bounded wait of 60 seconds. This allocate-stream-release pattern distributes GPU capacity fairly across many simultaneous users without dedicating hardware to any session; a lease reaper force-releases stalled generations and periodic GPU health polling excludes unreachable servers, ensuring fault-tolerant multi-user serving within the offline environment.

V. IMPLEMENTATION DETAILS

The framework is realized as a set of loosely coupled microservices communicating over localhost interfaces, with every model and database executed locally. This section details the concrete technology stack, the retrieval fusion algorithm, and the configuration governing the end-to-end pipeline, complementing the conceptual treatment in the Methodology.

A. Technology Stack

Table IV summarizes the core technologies and their roles in the pipeline. All components are selected for offline deployability, CPU or GPU efficiency on the available hardware, and

native support for the persistence and streaming requirements of the system.

The orchestration layer instantiates a pool of isolated worker processes for text extraction, Qdrant ingestion, and Neo4j ingestion. Results are written to disk before submission and replayed on restart, providing crash recovery without reprocessing completed checkpoints. All inter-service communication occurs over internal channels, and model checkpoints are bundled offline so that no telemetry or model download reaches external networks.

B. Hybrid Retrieval and Fusion Algorithm

Algorithm 1 formalizes the query-time pipeline. Given a user query, the system extracts entities, classifies query intent, and executes four retrieval pathways in parallel: proposition-level vector search, section-level vector search, graph traversal, and hybrid BM25. The candidate pools are merged, deduplicated, reranked, expanded, and greedily assembled into a grounded context for the local LLM.

Algorithm 1 Hybrid Retrieval, Fusion, and Reranking

Require: Query Q ; top- k limits k_v, k_g ; RRF constant $k_{\text{rrf}} = 60$

Ensure: Grounded answer R

- 1: $E_Q \leftarrow \text{GLiNER}(Q)$ ▷ entity extraction
- 2: $c \leftarrow \text{QueryClassify}(Q)$ ▷
spec/process/comparative/overview/general
- 3: $\vec{q} \leftarrow \text{embed}(Q)$ ▷ nomic 768-d
- 4: $P_v \leftarrow \text{QdrantSearch}(\vec{q}, \text{propositions})$
- 5: $S_v \leftarrow \text{QdrantSearch}(\vec{q}, \text{sections})$
- 6: $S_g \leftarrow \text{Neo4jTraverse}(E_Q)$ ▷ $O(d^k)$ traversal
- 7: $S_h \leftarrow \text{RRF}(S_{\text{BM25}}, S_v)$ ▷ k=60 fusion
- 8: $C \leftarrow \text{Merge}(P_v, S_v, S_g, S_h)$ ▷ deduplicate + confidence boost
- 9: $C \leftarrow \text{Rerank}(Q, C)$ ▷ Jina cross-encoder + MMR
- 10: $C \leftarrow \text{PageExpand}(C)$ ▷ small-to-big metadata expansion
- 11: $C_{\text{ctx}} \leftarrow \text{GreedySelect}(C, \text{value density}, \text{budget})$
- 12: $R \leftarrow \text{LLM}(Q, C_{\text{ctx}})$ ▷ controlled decoding **return** R

The algorithm is executed with adaptive retrieval depth: candidates are first retrieved at half depth, and re-retrieved with doubled limits when the top reranker score falls below a threshold of 0.15. Proposition-score fusion further boosts a section by $0.10 \times \text{boost_mult}$ times its best child-proposition score, bridging fine-grained and section-level retrieval.

C. Configuration and Hyperparameters

Table V summarizes the principal configuration parameters and their values. These values were selected to balance retrieval precision, context budget, and generation latency under the three-user-slot concurrency model.

D. Deployment and Multi-User Serving

The deployment runs entirely within an air-gapped network, with databases (Qdrant, Neo4j, MongoDB), the embedding and NER models, the reranker, and the LLMs co-located on the

TABLE V
KEY CONFIGURATION AND HYPERPARAMETERS

Parameter	Value
Embedding model	nomic-embed-text-v1.5, 768-d, 8,192 ctx
NER model	GLiNER multi-v2.1 (7 default + corpus labels)
Reranker	Jina Reranker v3, 131K ctx
RRF constant k_{rrf}	60
MMR trade-off	0.7 (score) / 0.3 (diversity)
Proposition-score boost	0.10
Adaptive depth factor	0.5, re-retrieve $\times 2$
Top reranker acceptance	0.15
Min. proposition length	8 words / 30 characters
Page expansion	nearest neighboring pages
GPU slots per model	up to 3 concurrent
FIFO wait bound	60 s

local GPU fleet. The GPU load-balancing service pools three RTX 5060 servers, each hosting multiple model slots; a slot is allocated to the server with the highest available capacity for the requested inference mode, tokens are streamed to the client in real time, and the slot is released on completion. A lease reaper force-releases stalled generations, periodic health polling excludes unreachable servers, and requests exceeding the 60-second bound are shed gracefully, ensuring fault-tolerant multi-user serving without dedicating hardware to any single session.

VI. THEORETICAL NOVELTY AND ALGORITHMIC CONTRIBUTIONS

The novelty of KRUTRIM RAG does not originate from any individual component, each explored independently in prior literature, but from integrating multiple retrieval, indexing, and knowledge representation paradigms into a unified architecture for secure, air-gapped, resource-constrained enterprise environments.

A. Synchronized Dual-Path Ingestion with Deterministic Metadata Alignment

Existing hybrid architectures construct vector indexes and knowledge graphs independently, aligning them at retrieval time through heuristic matching and introducing provenance ambiguity. KRUTRIM RAG instead generates embeddings and graph entities simultaneously from a shared metadata schema during ingestion: for every document fragment d_i , a metadata fingerprint $M_i = (\text{DocID}, \text{Source}, \text{Page}, \text{Section})$ is injected into both retrieval spaces: $\vec{v}_i \in \mathbb{R}^{768}$ in Qdrant and $n_i \in G(V, E)$ in Neo4j, so that $\vec{v}_i \leftrightarrow n_i$ via the shared fingerprint. This establishes structural consistency at ingestion time, enabling section-path-based alignment, provenance verifiability, and reduced semantic ambiguity during context assembly.

B. Resource-Aware Graph Traversal through Index-Free Adjacency

Neo4j’s IFA mechanism keeps traversal workload governed by the local neighborhood structure, $O(\bar{d}^k)$ for depth k and average branching factor $\bar{d}$, rather than total graph cardinality $|V|$. This reduces dependence on graph size, enables efficient multi-hop reasoning as repositories expand, and minimizes memory overhead from repeated global index lookups, preserving resources for embedding generation and LLM inference.

C. Automated Load-Balanced Parallel Knowledge Ingestion

Sequential ingestion pipelines become inefficient at enterprise scale. KRUTRIM RAG distributes documents $D = \{d_1, \dots, d_n\}$ across workers $W = \{W_1, \dots, W_m\}$, each independently performing extraction, preprocessing, chunking, metadata generation, and indexing. Whereas sequential execution costs $T_{\text{seq}} = \sum_{i=1}^n T_i$, parallel execution approaches $T_{\text{par}} \approx \frac{1}{m} \sum_{i=1}^n T_i$, yielding an approximately linear speedup proportional to available workers.

D. Heterogeneous Multi-Stream Retrieval Fusion

The framework integrates four retrieval pathways: proposition-level vector search, section-level vector search, graph traversal, and hybrid BM25, merging them through weighted fusion with confidence boosts for corroborated evidence, refined by cross-encoder reranking, MMR diversity selection, and proposition-score aggregation. This multi-granularity approach enables precise atomic-fact matching while preserving full section context.

VII. EXPERIMENTAL SETUP

A. Experimental Environment

The framework was evaluated in a fully offline, air-gapped environment. Tables VI and VII summarize the hardware and software configuration.

TABLE VI
HARDWARE CONFIGURATION

Component	Specification
CPU	Intel Xeon (multi-core)
RAM	64 GB DDR4
GPU	3× NVIDIA RTX 5060 (8 GB VRAM each)
Storage	1 TB NVMe SSD
Network	Air-gapped

B. Datasets

The evaluation covers three publicly benchmarked QA datasets, each stressing a different retrieval challenge relevant to a general-purpose RAG pipeline.

E-Manual [22] is a domain-specific QA dataset built from Samsung Electronics product manuals (TVs, appliances), designed for instructional/procedural question answering rather

TABLE VII
SOFTWARE CONFIGURATION

Component	Version / Specification
Vector DB	Qdrant (HNSW, cosine, local)
Graph DB	Neo4j (5-layer property graph)
Central DB	MongoDB (local)
Embedding	nomic-embed-text-v1.5 (768-d)
NER	GLiNER (gliner_multi-v2.1)
Reranker	Jina Reranker v3 (131K context)
LLMs	Mistral 7B, Qwen 3 14B, Gemma 3
Parser	Docling
Workers	Multi-process + GPU-based

than open-domain factoid QA - answers are typically embedded in step-by-step technical instructions. In our evaluation subset it consists of 102 indexed chunks and 66 unique queries, making it the smallest and most domain-narrow corpus tested.

HotpotQA [23] is a widely used open-domain, multi-hop QA benchmark built on Wikipedia, requiring a retriever to surface multiple supporting passages that must be combined to answer a single question. Our evaluation subset contains 1,550 chunks and 390 queries, and serves as a stress test for whether graph-based entity linking improves multi-hop evidence retrieval over dense retrieval alone.

TAT-QA [24] is a hybrid tabular-and-textual QA benchmark drawn from real-world financial reports, requiring both textual comprehension and numerical reasoning over tables. It is the largest evaluation subset by query count (1,272 chunks, 1,668 queries) and tests retrieval robustness on structurally heterogeneous (text + table) content.

Together, the three datasets span three distinct retrieval regimes: narrow-domain procedural text (E-Manual), multi-hop open-domain text (HotpotQA), and hybrid tabular/financial text (TAT-QA).

TABLE VIII
RETRIEVAL PERFORMANCE ON E-MANUAL

Metric	Dense Retrieval	Hybrid Graph Retrieval
R@1	0.6061	0.5783
R@5	0.9823	0.9596
NDCG@5	0.9153	0.8876
Hit@5	1.0000	0.9848
MRR	0.9217	0.8944

C. Evaluation Metrics

The evaluation spans retrieval quality, generation fidelity, and system-level efficiency. *Retrieval metrics*: Recall@K, Precision@K, Mean Reciprocal Rank (MRR), and Normalized Discounted Cumulative Gain (nDCG). *Generation quality metrics*: faithfulness, context relevance, and hallucination rate. *System performance metrics*: ingestion throughput, query latency, and CPU/memory utilization.

TABLE IX
RETRIEVAL PERFORMANCE ON HOTPOTQA

Metric	Dense Retrieval	Hybrid Graph Retrieval
R@1	0.4970	0.4970
R@5	0.9688	0.9688
NDCG@5	0.9642	0.9638
Hit@5	1.0000	1.0000
MRR	0.9893	0.9893

TABLE X
RETRIEVAL PERFORMANCE ON TAT-QA

Metric	Dense Retrieval	Hybrid Graph Retrieval
R@1	0.5093	0.5066
R@5	0.6374	0.6382
NDCG@5	0.6034	0.6015
Hit@5	0.6775	0.6787
MRR	0.6171	0.6140

VIII. RESULTS AND EVALUATION

A. Knowledge Base Construction

The ingestion pipeline transformed the corpus into 768-dimensional embeddings indexed by HNSW in Qdrant, with a dual-collection architecture of approximately 3,800 propositions and 3,000 context-preserving sections; ingestion throughput scaled approximately linearly with worker count.

B. Knowledge Graph Representation and Persistence

The graph construction pipeline produced a structured knowledge graph in Neo4j; for the 900+ page benchmark document this comprised approximately 9,000 nodes across five layers, comprising approximately 19,500 entity mentions and 2,700 specification nodes, with 91.6% knowledge coverage. Graph construction completed in approximately 12 minutes (727s) under the parallel configuration, down from 2,294s sequentially. The graph was validated using Cypher queries and supports entity-centric retrieval and relationship exploration, improving contextual understanding and retrieval explainability. MongoDB serves as the central persistence store, managing user accounts, job lifecycle states (queued, extracting, extraction_done, ingesting, completed, extraction_failed, ingestion_failed), chat histories, and document provenance metadata; the MongoDB-backed queue dispatches jobs sequentially, and checkpoint files enable resume-on-failure, retrying only the incomplete stage without reprocessing completed checkpoints. The user interface supports document management, metadata filtering, query submission, and response visualization, with PDF.js-based in-browser document viewing and direct navigation to source pages from retrieved results.

C. Retrieval Performance

The retrieval performance of the framework was compared across the vector-only, graph-only, and hybrid retrieval

strategies. The hybrid approach consistently outperformed standalone strategies, and the full pipeline (with reranking, MMR diversity selection, page expansion, and proposition-score fusion) achieved the strongest recall while maintaining top precision, confirming that semantic and relational representations capture complementary signals.

D. Scalability and Ingestion Performance

Ingestion time was measured under sequential and parallel configurations on the 900+ page, four-document corpus yielding approximately 3,000 text chunks and 3,800 atomic propositions. The GPU-accelerated configuration (RTX 5060 for GLiNER NER, five embedding workers) reduced total ingestion time by 65%, from 5,245s to 1,821s:

- **Document extraction** improved 2.5x (645s $\rightarrow$ 261s); the sequential baseline was dominated by rate-limit and connection timeouts in worker-to-server communication, eliminated through parallel worker execution.
- **Qdrant embedding ingestion** improved 2.8x (2,306s $\rightarrow$ 833s); parallel workers reduced per-proposition embedding time for nomic-embed-text-v1.5 from $\sim$ 1.3s to $\sim$ 0.6s.
- **Neo4j graph ingestion** improved 3.2x (2,294s $\rightarrow$ 727s); GPU acceleration of the 300M+ parameter GLiNER model prevented six-minute batch processing stalls.
- **Knowledge coverage** remained consistent at 91.6% across both configurations.

E. Context Quality

Generation quality was evaluated across configurations; the full pipeline achieved the highest faithfulness and context relevance while attaining the lowest hallucination rate, with reranking and richer context assembly yielding progressively stronger grounding for generated responses.

IX. ABLATION STUDY

An ablation analysis evaluated the contribution of individual system components. Removing graph retrieval caused the largest degradation in precision, confirming the importance of structured knowledge for grounded responses; cross-encoder reranking and proposition-score fusion each contributed measurable precision gains, while the removal of MMR diversity increased precision but significantly decreased recall. Page expansion did not affect precision but preserved context quality, and removing the hybrid BM25 pathway slightly reduced overall retrieval effectiveness.

X. FAILURE ANALYSIS

Qualitative analysis of failure cases reveals systematic limitations. GLiNER entity extraction fails for a notable share of queries, typically procedural or comparative questions without explicit named entities, causing fallback to vector-only retrieval. Regex-based specification extraction (numeric-plus-unit patterns) exhibits a small but measurable false positive rate in manually reviewed specifications. Graph traversal beyond a shallow depth $k = 2$ degrades measurably without

confidence-weighted edge pruning. The proposition length filter discards a small proportion of propositions, including legitimate compact specifications (e.g., “Thrust: 799 kN”). Cross-document reasoning is limited without explicit comparison patterns, as the graph hierarchy creates no cross-document edges. The regex query classifier is accurate on held-out queries, with most confusion occurring between overview and general questions and between process and general questions; cold-start latency is markedly higher on the first query after startup due to lazy model loading, versus a lower steady-state latency.

Question quality varies systematically by question type. Specification lookup questions, which map directly to regex-extracted numeric-plus-unit patterns, are answered with high reliability and near-complete entity extraction, because the governing entities are explicitly present in the text. Overview questions also perform well, since their coarse-grained nature tolerates partial retrieval and is well served by the hierarchical section context. In contrast, comparative analysis questions are the most error-prone: they require named entities from both sides of the comparison, and the graph hierarchy creates no cross-document edges, so evidence for one side frequently remains unrecovered, lowering faithfulness of the generated answer. Process explanation questions suffer from weaker entity extraction because procedural content is often phrased without explicit named entities, causing fallback to vector-only retrieval and a loss of relational grounding. A further limitation concerns tabular content: when a question targets data stored in a table, the model frequently fails to produce a proper answer, because table rows are flattened into linearized text and structured column relationships are not preserved during retrieval; in contrast, questions grounded in paragraph text are answered mostly correctly, since paragraphs retain their original structure and semantics throughout the pipeline. General questions sit between these extremes, occasionally confused with overview or process intent by the regex classifier, but otherwise answered acceptably. These per-type differences align directly with the failure modes described above.

XI. DISCUSSION

The experimental findings provide strong evidence for the effectiveness of KRUTRIM RAG. Integration of semantic vector retrieval and graph-based reasoning consistently improved retrieval quality by combining semantic proximity with relational structure, confirming that semantic and relational representations capture complementary signals. Synchronized metadata alignment improved consistency by identifying sections independently confirmed through both channels, reducing hallucination rates through verifiable grounding. The load-balanced ingestion architecture achieved a 2.9x end-to-end speedup (87.4 minutes to 30 minutes and 21 seconds for the 900+ page document) while maintaining 91.6% knowledge coverage; cross-encoder reranking and proposition-score fusion added measurable precision gains, and pipeline refinements (query classification, hybrid BM25+vector fusion, MMR diversity, adaptive retrieval depth, greedy context allo-

cation, and dynamic prompts) collectively enhanced answer quality. Finally, the complete elimination of cloud dependencies across all components demonstrates that high-quality RAG can be achieved entirely within air-gapped environments without sacrificing retrieval effectiveness or contextual relevance.

XII. FUTURE WORK

Future work will focus on learning-based retrieval fusion, multi-model RAG architectures for collaborative reasoning, end-to-end trainable RAG pipelines, and dynamic knowledge updating for incremental ingestion without full re-indexing. Advanced query understanding (expansion, decomposition, conversational context) and deeper multi-hop graph reasoning will improve complex-question handling. The evaluation framework will be extended along several axes: (i) standardized retrieval benchmarks such as BEIR and MIRACL for comparable quality assessment; (ii) question-type-specific quality analysis, including entity-extraction success and faithfulness disaggregated by query category; (iii) systematic computation of nDCG, MRR, faithfulness, and hallucination rates with explainable attribution to retrieved evidence; (iv) human evaluation of response quality and retrieval explainability with domain experts; and (v) robustness studies measuring degradation under adversarial queries, OCR noise, and cross-document reasoning demands. In particular, tabular question answering will be strengthened by preserving structured table semantics during ingestion and retrieval, for example through row-and-column-aware embeddings, table-aware chunking, and structured cell-level retrieval, rather than relying on flattened linearized text; paragraph-grounded questions are already answered mostly correctly, but table-grounded questions require this structure-preserving treatment. Distributed retrieval architectures, multimodal integration of images and diagrams, and graph optimization (dynamic partitioning, caching) will also be explored.

XIII. CONCLUSION

This work presented KRUTRIM RAG, a fully offline hybrid Retrieval-Augmented Generation framework integrating semantic vector retrieval, graph-based reasoning, entity-aware query expansion, and neural reranking in a unified pipeline, addressing traditional RAG systems' reliance on a single retrieval paradigm and incomplete contextual grounding. The framework introduces four key contributions: synchronized dual-path ingestion with deterministic metadata alignment, resource-aware graph traversal through Index-Free Adjacency, automated load-balanced parallel ingestion with near-linear speedup, and heterogeneous multi-stream retrieval fusion combining semantic search, graph traversal, hybrid BM25, cross-encoder reranking, and proposition-score fusion, establishing a privacy-preserving, scalable, and context-aware retrieval architecture for enterprise-grade knowledge management and question answering in fully offline environments.

Experimental evaluation on aerospace technical documents demonstrated that the hybrid approach significantly outper-

forms standalone strategies, with the full pipeline achieving high precision, recall, and faithfulness at low latency, and the ablation study confirming graph retrieval as the largest individual improvement. All databases, embedding models, NER, rerankers, and LLMs operate exclusively on local infrastructure with zero cloud dependencies, demonstrating that high-quality RAG can be achieved within fully air-gapped environments.

REFERENCES

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in NeurIPS*, vol. 33, pp. 9459–9474, 2020.
- [2] J. Gao, Y. Xiong, P. Zhang, H. Wang, and J. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [3] H. Touvron et al., "Llama 3: Open Foundation and Fine-Tuned Chat Models," *Meta AI Technical Report*, 2024.
- [4] Neo4j Inc., "Neo4j Graph Database Platform," 2026. <https://neo4j.com/docs/>
- [5] Qdrant Team, "Qdrant: Vector Search Engine," 2026. <https://qdrant.tech/documentation/>
- [6] M. Samoshyn, A. Khromov, and Qdrant Team, "Efficient Approximate Nearest Neighbor Search Using HNSW in Vector Databases," *Qdrant Technical Documentation*, 2024.
- [7] Y. Malkov and D. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
- [8] G. Angles, "The Property Graph Database Model," *Proc. ACM Symposium on Applied Computing*, pp. 390–395, 2018.
- [9] A. Hogan et al., "Knowledge Graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.
- [10] A. Zaratian, N. Tomeh, P. Holat, and V. Charnois, "GLiNER: Generalist and Lightweight Model for Named Entity Recognition," *Proc. NAACL*, 2024.
- [11] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," *SIGIR*, pp. 39–48, 2020.
- [12] G. Cormack, C. Clarke, and S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," *SIGIR*, pp. 758–759, 2009.
- [13] F. Wang, Y. Li, and H. Xiao, "jina-reranker-v3: Last but Not Late Interaction for Listwise Document Reranking," *arXiv preprint arXiv:2509.25085*, 2025.
- [14] Nomic AI, "Nomic Embed Text v1.5 Technical Report," 2024. <https://docs.nomic.ai/platform/embeddings-and-retrieval/text-embedding>
- [15] C. Auer et al., "Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion," *arXiv preprint arXiv:2408.09869*, 2024.
- [16] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," *Proc. SIGIR*, pp. 335–336, 1998.
- [17] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [18] B. Sarmah et al., "HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction," *arXiv preprint arXiv:2408.04948*, 2024.
- [19] C. Min et al., "Towards Practical GraphRAG: Efficient Knowledge Graph Construction and Hybrid Retrieval at Scale," *arXiv preprint arXiv:2507.03226*, 2025.
- [20] Y. Cao et al., "LEGO-GraphRAG: Modularizing Graph-based Retrieval-Augmented Generation for Design Space Exploration," *Proc. VLDB Endowment*, vol. 18, no. 10, pp. 3269–3283, 2025.
- [21] P. P. Mishra et al., "A Systematic Framework for Enterprise Knowledge Retrieval: Leveraging LLM-Generated Metadata to Enhance RAG Systems," in *2026 IEEE Conf. on Artificial Intelligence (CAI)*, 2026.
- [22] A. Nandy, S. Sharma, S. Maddhashiya, K. Sachdeva, P. Goyal, and N. Ganguly, "Question Answering over Electronic Devices: A New Benchmark Dataset and a Multi-Task Learning based QA Framework,"

in *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, pp. 4600–4609.

- [23] Z. Yang et al., “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,” in *Proc. EMNLP*, 2018.
- [24] F. Zhu et al., “TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance,” in *Proc. ACL-IJCNLP*, 2021.